

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 3
Dynamic Programming

Problem Statement: 1

Problem – Coin Change

1. Dynamic Programming Formulation:

- **Subproblem/State Definition:** Let $DP[a]$ denote the minimum number of coins required to form a target amount a , where $0 \leq a \leq A$.

- **Base Cases:**

$$DP[0] = 0 \quad (0 \text{ coins are needed to make an amount of } 0)$$

$$DP[a] = \infty \quad \text{for all } a > 0 \quad (\text{initialized to infinity})$$

- **State Transition (Recurrence Relation):**

$$DP[a] = 1 + \min_{\substack{c \in \text{coins} \\ c \leq a}} DP[a - c]$$

If $DP[A] = \infty$, then the target amount cannot be formed using the given denominations.

Code:

```
1 def coin_change_dp(coins, amount):
2     """Bottom-up Dynamic Programming with coin combination reconstruction."""
3     dp = [0] + [float('inf')] * amount
4     parent = [-1] * (amount + 1)
5
6     for a in range(1, amount + 1):
7         for c in coins:
8             if c <= a and dp[a - c] + 1 < dp[a]:
9                 dp[a] = dp[a - c] + 1
10                parent[a] = c
11
12     if dp[amount] == float('inf'):
13         return -1, []
14
15     # Reconstruct optimal combination
16     combination, curr = [], amount
17     while curr > 0:
18         c = parent[curr]
19         combination.append(c)
20         curr -= c
21
22     return dp[amount], combination
23
24
25 def coin_change_rec(coins, amount):
```

```

26 """Naive recursive approach without memoization."""
27 if amount == 0:
28     return 0
29 min_coins = float('inf')
30 for c in coins:
31     if c <= amount:
32         res = coin_change_rec(coins, amount - c)
33         if res != float('inf'):
34             min_coins = min(min_coins, 1 + res)
35 return min_coins
36
37
38 def main():
39     # PDF Example: coins = {1, 3, 4}, target amount = 6
40     coins = [1, 3, 4]
41     amount = 6
42
43     min_coins, combo = coin_change_dp(coins, amount)
44     print(f"Denominations : {coins}")
45     print(f"Target Amount : {amount}")
46     print(f"Minimum Coins : {min_coins}")
47     print(f"Combination   : {' + '.join(map(str, combo))} = {amount}")
48
49     # Boundary Cases
50     print("\n--- Boundary Cases ---")
51     print(f"Target 0: min_coins = {coin_change_dp(coins, 0)[0]}, combination = {
52         coin_change_dp(coins, 0)[1]}")
53     print(f"Unreachable Target (coins=[2, 4], amount=7): {coin_change_dp([2, 4], 7)}")
54
55 if __name__ == "__main__":
56     main()

```

Output:

```

Denominations : [1, 3, 4]
Target Amount : 6
Minimum Coins : 2
Combination   : 3 + 3 = 6

--- Boundary Cases ---
Target 0: min_coins = 0, combination = []
Unreachable Target (coins=[2, 4], amount=7): (-1, [])

```

Complexity Comparison (DP vs. Naive Recursion):

Approach	Time Complexity	Auxiliary Space
Bottom-Up DP	$O(n \cdot A)$	$O(A)$ (DP and parent tracking tables)
Naive Recursive	$O(n^A)$ (Exponential)	$O(A)$ (Maximum recursion call stack depth)

Table 1: Theoretical complexity comparison for Coin Change ($n = |\text{coins}|$, $A = \text{amount}$).

Empirical Execution Time Experimentation:

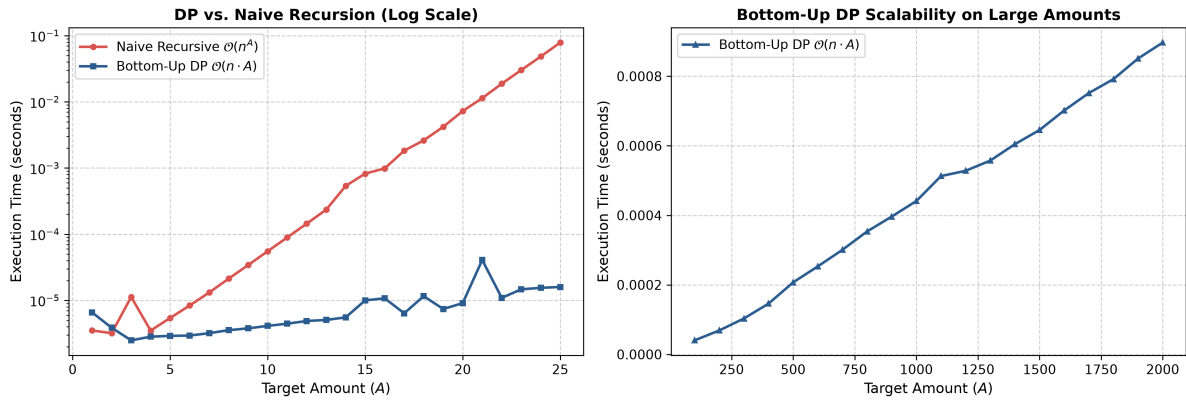


Figure 1: Execution time comparison: (Left) Log-scale execution time comparing DP $O(n \cdot A)$ vs. Naive Recursion $O(n^4)$ for $A \in [1, 25]$; (Right) Linear scalability of Bottom-Up DP for large amounts up to $A = 2000$.

Analysis:

1. Greedy Strategy Evaluation: A greedy strategy selects the largest available denomination $c \leq$ remaining amount at every step. While this produces the optimal minimum number of coins for canonical currency systems (e.g., standard denominations), it does **not** always guarantee an optimal solution for arbitrary coin systems.

2. Counterexample (from PDF): Consider coin denominations $\{1, 3, 4\}$ and target amount 6:

- **Greedy Strategy:**

1. Step 1: Chooses denomination 4 (Remaining amount: $6 - 4 = 2$).
2. Step 2: Chooses denomination 1 (Remaining amount: $2 - 1 = 1$).
3. Step 3: Chooses denomination 1 (Remaining amount: $1 - 1 = 0$).
4. **Total coins selected by Greedy:** 3 coins ($4 + 1 + 1 = 6$).

- **Dynamic Programming (Optimal):**

1. Evaluates subproblems $DP[6 - 3] + 1 = DP[3] + 1 = 1 + 1 = 2$.
2. **Total coins selected by DP:** 2 coins ($3 + 3 = 6$).

3. Why Dynamic Programming Succeeds where Greedy Fails: The greedy approach makes a short-sighted local optimum (picking 4) without considering how that choice restricts future decisions. In contrast, Dynamic Programming examines all valid coin transitions $DP[a - c] + 1$ across every subproblem amount a . By utilizing **optimal substructure** and memoizing optimal subproblem solutions, DP guarantees finding the global minimum.

Problem Statement: 2

Problem – Employee Training Planning

1. Dynamic Programming Formulation (0/1 Knapsack):

- **Problem Formulation:** Given n training modules with costs c_i and benefits b_i , and a maximum training budget B , select a subset of modules to maximize total benefit such that total cost $\leq B$. Each module can be selected at most once.
- **State Definition:** Let $DP[i][w]$ denote the **maximum achievable benefit** considering a subset of the first i modules under budget constraint w , where $0 \leq i \leq n$ and $0 \leq w \leq B$.
- **Base Cases:**

$$DP[0][w] = 0 \quad \text{for all } 0 \leq w \leq B \quad (0 \text{ benefit if no modules are chosen})$$

$$DP[i][0] = 0 \quad \text{for all } 0 \leq i \leq n \quad (0 \text{ benefit if budget is 0})$$

- **State Transition (Recurrence Relation):**

$$DP[i][w] = \begin{cases} DP[i-1][w] & \text{if } c_i > w \\ \max(DP[i-1][w], DP[i-1][w - c_i] + b_i) & \text{if } c_i \leq w \end{cases}$$

Code:

```

1 def employee_training_01_dp(modules, budget):
2     """
3     Bottom-up 0/1 Dynamic Programming for Employee Training Planning.
4     modules: list of tuples (name, cost, benefit)
5     budget: maximum training budget B
6     """
7     n = len(modules)
8     dp = [[0] * (budget + 1) for _ in range(n + 1)]
9
10    for i in range(1, n + 1):
11        name, cost, benefit = modules[i - 1]
12        for w in range(budget + 1):
13            if cost <= w:
14                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - cost] + benefit)
15            else:
16                dp[i][w] = dp[i - 1][w]
17
18    # Reconstruct selected modules
19    selected_modules = []
20    w = budget
21    total_cost = 0
22    for i in range(n, 0, -1):
23        if dp[i][w] != dp[i - 1][w]:
24            name, cost, benefit = modules[i - 1]
25            selected_modules.append(name)
26            total_cost += cost
27            w -= cost
28
29    selected_modules.reverse()
30    return dp[n][budget], total_cost, selected_modules

```

```
31
32
33 def main():
34     # PDF Module Data
35     modules = [
36         ("M1", 4, 7),
37         ("M2", 6, 10),
38         ("M3", 5, 8),
39         ("M4", 3, 5)
40     ]
41     budget = 10
42
43     max_benefit, total_cost, selected = employee_training_01_dp(modules, budget)
44
45     print("=== PDF Example Solution (Budget B = 10) ===")
46     print(f"Available Modules : {modules}")
47     print(f"Maximum Benefit   : {max_benefit}")
48     print(f"Total Cost           : {total_cost} / {budget}")
49     print(f"Selected Modules    : {selected}")
50
51     # Boundary Cases
52     print("\n--- Boundary Cases ---")
53     print(f"Budget 0 : {employee_training_01_dp(modules, 0)}")
54     print(f"Budget 2 (Below minimum module cost): {employee_training_01_dp(modules, 2)}")
55
56
57 if __name__ == "__main__":
58     main()
```

Output:

```
=== PDF Example Solution (Budget B = 10) ===
Available Modules : [('M1', 4, 7), ('M2', 6, 10), ('M3', 5, 8), ('M4', 3, 5)]
Maximum Benefit   : 17
Total Cost        : 10 / 10
Selected Modules  : ['M1', 'M2']

--- Boundary Cases ---
Budget 0 : (0, 0, [])
Budget 2 (Below minimum module cost): (0, 0, [])
```

Complexity Analysis:

Implementation	Time Complexity	Auxiliary Space
2D Bottom-Up DP	$O(n \cdot B)$	$O(n \cdot B)$ (Table of size $(n + 1) \times (B + 1)$)
1D Space-Optimized	$O(n \cdot B)$	$O(B)$ (Single 1D array traversed backwards)

Table 2: Complexity breakdown for Employee Training Planning (n = number of modules, B = budget).

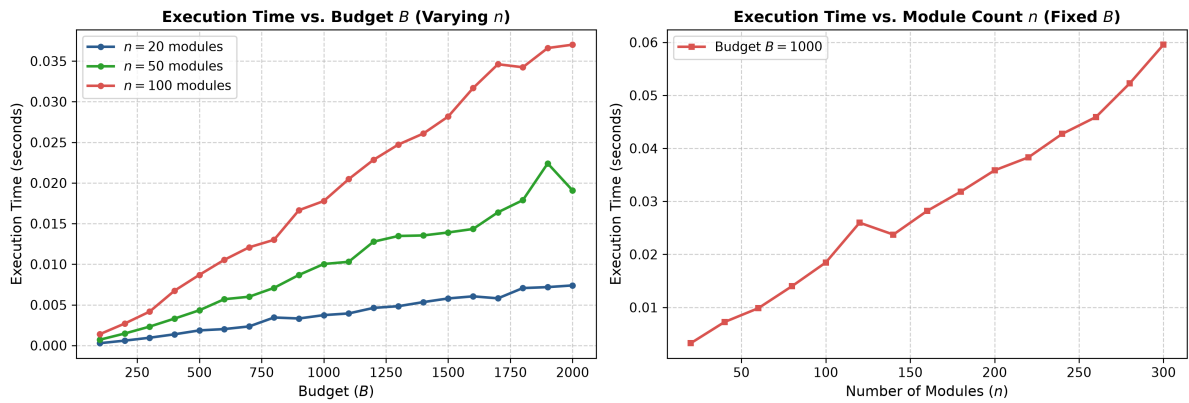
Empirical Execution Time Experimentation:

Figure 2: Empirical execution time analysis: (Left) Linear scaling with budget B across different module counts $n \in \{20, 50, 100\}$; (Right) Linear scaling with module count n for fixed budget $B = 1000$.

Analysis:

1. Problem Formulation under Module Repetition (Unbounded Knapsack): When training modules may be selected **more than once**, the constraint transitions from a 0/1 decision to an **Unbounded Knapsack** problem.

2. Mathematical Formulation & Recurrence Changes:

- **0/1 Constraint:** Once module i is chosen, the remaining budget must be filled using only the remaining $i - 1$ modules:

$$DP[i][w] = \max \left(DP[i-1][w], DP[i-1][w - c_i] + b_i \right)$$

- **Unbounded Modification:** Module i can be repeatedly selected from the current state $w - c_i$:

$$DP[w] = \max_{\substack{1 \leq i \leq n \\ c_i \leq w}} \left(DP[w - c_i] + b_i \right), \quad \text{with } DP[0] = 0$$

3. Effect on Selection Behavior & Complexity:

- **Selection Behavior:** The algorithm can repeatedly exploit the module with the highest benefit-to-cost efficiency ratio $\frac{b_i}{c_i}$.

- For the PDF modules: $M_1(7/4 = 1.75)$, $M_2(10/6 \approx 1.67)$, $M_3(8/5 = 1.60)$, $M_4(5/3 \approx 1.67)$.
 - For budget $B = 12$, the 0/1 optimal solution selects $M_1 + M_3 + M_4$ with total Benefit = **20**.
 - In contrast, the Unbounded solution selects $M_1 \times 3$ with total Benefit = **21** (3×7), achieving strictly higher benefit.
- **Space Complexity:** Reduces from $O(n \cdot B)$ in 2D to $O(B)$ in 1D array formulation.
 - **Time Complexity:** Remains $O(n \cdot B)$, requiring n transitions for each budget step $1 \dots B$.